

```

/*
Exercice n°1 :
Ecrire en C les sous-programmes suivants :
- lire : permet de lire une variable entière strictement positive
- maximum : permet de déterminer le maximum de deux variables entières
Le programme principal permet de lire trois variables entières a, b et c et d'afficher la
valeur maximale.
*/

```

```

#include <stdio.h>

//Définition de fonctions globales
int lire()
{
    int a;
    do{
        scanf("%d",&a);
    }while(a<=0);
    return(a);
}
//=====
int maximum(int a,int b)
{
    int max;
    if(a>b)
        max=a;
    else
        max=b;
    return max;
}
//=====
int main() {
    int a,b,c,m;
    printf("\n====EXERCICE N 1 :==== \n\n");
    printf("Saisie de 3 entiers strictement positif\n");
    //Appel des fonctions lire et maximum
    a=lire();
    b=lire();
    c=lire();
    m=maximum(a,b);
    m=maximum(m,c);
    printf("%d est le plus grand entier\n\n",m);
    return 0;
}

```

====EXERCICE N 1 :====

Saisie de 3 entiers strictement positif
99
5
14
99 est le plus grand entier

```

//Définition de fonctions après la fonction principale main()
#include <stdio.h>
int main() {
    int a,b,c,m;
    //Il faut déclarer les fonctions (donner les prototypes)
    int lire();
    int maximum(int, int);
    //
    printf("\n====EXERCICE N 1 :==== \n\n");
    printf("Saisie de 3 entiers strictement positif\n");
    //Appel des fonctions lire et maximum
    a=lire();

```

```

    b=lire();
    c=lire();
    m=maximum(a,b);
    m=maximum(m,c);
    printf("%d est le plus grand entier\n\n",m);
    return 0;
}
//=====
int lire()
{
    int a;
    do{
        scanf("%d",&a);
    }while(a<=0);
    return(a);
}
//=====
int maximum(int a,int b)
{
    int max;
    if(a>b)
        max=a;
    else
        max=b;
    return max;
}

```

```

#include <stdio.h>

//Passage par adresse de a dans la fonction lire
void lire(int* a)
{
    do{
        scanf("%d",a);
    }while(*a<=0);
}
//=====
int maximum(int a,int b)
{
    int max;
    if(a>b)
        max=a;
    else
        max=b;
    return max;
}
//=====
int main() {
    int a,b,c,m;
    printf("\n====EXERCICE N 1 :==== \n\n");
    printf("Saisie de 3 entiers strictement positif\n");
    /*Appel des fonctions lire et maximum (remarquez les paramètres effectifs dans le
    passage par adresse)*/
    lire(&a);
    lire(&b);
    lire(&c);
    m=maximum(a,b);
    m=maximum(m,c);
    printf("%d est le plus grand entier\n\n",m);
    return 0;
}

```

```

/*
Exercice n °2 :
Ecrire un programme en C permettant de calculer le nombre de combinaisons de p éléments
parmi un ensemble de n éléments, note C(n,p)
On rappelle que :  $C(n,p) = n! / (p!(n-p)!)$ , avec  $n! = \text{produit}(n-i)$  avec i de 0 à (n-1)
La solution doit comporter au moins les sous-programmes permettant de :
- Lire les variables entières p et n avec  $0 < p \leq n$ 
- Calculer le factoriel d'une variable entière
Le calcul du résultat final peut être inclus dans le programme principal ou à partir d'un
sous-programme.
*/
/*
Remarque :
Le résultat d'une combinaison est toujours un nombre entier, et non un nombre réel
quelconque.
*/

```

```

#include <stdio.h>

//fonction pour lire les variables entieres p et n avec 0<p<=n
void lire(int* p, int* n)
{
    do{
        scanf("%d%d",p,n);
    }while((*p<=0)||(*p>*n));
}

//=====
/*int factoriel(int a)
{
    int i,f=1;
    for(i=2;i<=a;i++)
        f=f*i;
    return f;
}*/
int factoriel(int a)
{
    int i,f=1;
    for(i=0;i<=(a-1);i++)
        f=f*(a-i);
    return f;
}

//=====
int main()
{
    int p,n,C;
    printf("\n====EXERCICE N 2 :==== \n\n");
    printf("Donner p puis n avec (0<p<=n)\n");
    lire(&p,&n);
    C=factoriel(n)/(factoriel(p)*factoriel(n-p));
    printf("C(%d,%d) = %d \n\n",p,n,C);
    return 0;
}

```

```

====EXERCICE N 2 :====

Donner p puis n avec (0<p<=n)
5
9
C(5,9) = 126

```

```

/*
Exercice n°3 :
Ecrire en C un programme qui permet de lire une sequence binaire dont la taille ne depasse
pas 8 bits,puis il affiche le resultat de la conversion en decimale.
Exemple :  $N=11001 = (1*2^0) + (0*2^1) + (0*2^2) + (1*2^3) + (1*2^4) = 25$ 
La solution a proposer doit envisager des sous programmes pour :
- Verifier que la suite saisie est binaire et de taille <=8
- Calcul de la puissance
*/

```

```

#include <stdio.h>
//=====
int Verifier(int* N,int* nbchiff) {
    int chiff,valid=1;
    *nbchiff=0;
    *N = 0;
    scanf("%d",&chiff);
    while(chiff!=-1){

        if(chiff!=0&&chiff!=1)
        {
            valid=0;
        }
        else
        {
            *N = (*N)*10+chiff;
            (*nbchiff)++;
        }
        scanf("%d",&chiff);
    }
    if(*nbchiff>8)
        valid=0;

    return valid;
}
//=====
int conversion(int N,int nbchiff) {
    int Ndec=0,chiff,puiss=1,i;
    int nb=nbchiff;
    do{
        //____pow(2,nbchiff)____
        puiss=1;
        for(i=0;i<nbchiff-nb;i++)
            puiss=2*puiss;

        //_____
        chiff=N%10;
        Ndec = Ndec + chiff*puiss;
        nb--;
        N=N/10;
    }while(N!=0);
    return Ndec;
}
//=====
int main()
{
    int seq,N,nb,dec;
    printf("\n====EXERCICE N 3 :==== \n\n");
    printf("Entrez une sequence binaire (terminer par -1) : \n");
    do{
        seq=Verifier(&N,&nb);
        if(seq==0)
            printf("Sequence invalide ==> ressaisir\n");
    }while(seq==0);
    printf("La sequence binaire est : %d\n",N);
    dec=conversion(N,nb);
    printf("En decimal = %d \n",dec);

    return 0;
}

```

====EXERCICE N 3 :====

```

Entrez une sequence binaire (terminer par -1) :
1
0
1
-1
La sequence binaire est : 101
En decimal = 5

```

```

/*
Solution2 Exercice n°3 avec fonction puissance de 2
*/

```

```

#include <stdio.h>
/*Verifier que la suite saisie est binaire (la séquence binaire jusqu'à -1) et de taille
<=8*/
int Verifier(int* N, int* nbchiff) {
    int chiff, valid = 1;
    *nbchiff = 0;
    *N = 0;

    scanf("%d", &chiff);
    while (chiff != -1) {
        if (chiff != 0 && chiff != 1) {
            valid = 0;
        } else {
            *N = (*N) * 10 + chiff;
            (*nbchiff)++;
        }
        scanf("%d", &chiff);
    }
    if (*nbchiff > 8)
        valid = 0;
    return valid;
}

//puissance de 2 ==> pow(2,n)
int puissance2(int n) {
    int result = 1;
    for (int i = 0; i < n; i++) {
        result *= 2;
    }
    return result;
}

// Fonction pour convertir un nombre binaire (donné comme entier) en décimal
int convertir(int N, int nbchiff) {
    int Ndec = 0, chiff;
    for (int i = 0; i < nbchiff; i++) {
        chiff = N % 10;
        Ndec += chiff * puissance2(i);
        N /= 10;
    }
    return Ndec;
}

//=====
int main() {
    int N, nb, seq, dec;
    printf("\n====EXERCICE N 3 :==== \n\n");
    printf("Entrez une sequence binaire (terminer par -1, max 8 chiffres) :\n");
    do {
        seq = Verifier(&N, &nb);
        if (!seq)
            printf("Sequence invalide ==> Ressaisir\n");
    } while (!seq);
    printf("La sequence binaire est : %d\n", N);
    dec = convertir(N, nb);
    printf("La valeur decimale est: %d\n\n", dec);
    return 0;
}

```

```

Entrez une sequence binaire (terminer par -1, max 8 chiffres) :
1
1
0
0
1
0
0
0
0
0
-1
Sequence invalide ==> Ressaisir
1
1
0
0
1
-1
La sequence binaire est : 11001
La valeur decimale est: 25

```

```

/*
Solution Exercice n°3 proposé par votre amie Dhakira Guirat
*/

```

```

#include <stdio.h>
bool verif(int x){
    int t=0;
    bool ok=true;

    while (x!=0&&ok){
        t+=1;
        ok=((x%10==0)|| (x%10==1)) && t<=8;
        x=x/10;
    }
    return ok;
}
int main() {
    int x,nb,i,s;

    do {
        printf("donner un nombre binaire : ");
        scanf("%d",&x);
    }while(verif(x)==false);
    //printf("Le nombre = %d\n",x);
    //=====
    nb=x; //garder la valeur de x pour l'affichage
    i=1;
    s=0;
    while(x!=0){
        s+=(x%10)*i;
        i=i*2;
        x=x/10;
    }
    printf("Le codage en decimale du %d est %d",nb,s);
    return 0;
}

```

```

donner un nombre binaire : 111100001
donner un nombre binaire : 11110000
Codage en decimale de 11110000 est 240

```

```

donner un nombre binaire : 11001
Codage en decimale de 11001 est 25

```

```

/*
Exercice n°4
Ecrire en C un programme qui permet de lire que les entiers qui s'écrivent sous la forme
2^n, et de les afficher sous forme de puissance de 2.
Exemple : Pour a= 64, le programme affiche : 64 = 2 puissance 6
La solution à proposer doit utiliser inclure des sous-programmes
*/

```

```

#include <stdio.h>
// Fonction pour calculer l'exposant de 2 pour un nombre donné
int puissance2(int n) {
    int result = 1;
    for (int i = 0; i < n; i++) {
        result *= 2;
    }
    return result;
}
// =====
int Tester(int n) {
    int x,i=0;

    do{
        x=puissance2(i);
        i++;
    }while(x<n);
}

```

```

    if (x==n) {
        printf("%d = 2 puissance %d\n", n, i-1);
    } else {
        printf("%d n'est pas une puissance de 2.\n", n);
    }
    return (x==n);
}
// =====
int main() {

    int nombre;
    printf("\n====EXERCICE N 4 :==== \n\n");
    do{
        printf("Entrer un entier : ");
        scanf("%d", &nombre);
    }while(!Tester(nombre));

    return 0;
}

```

====EXERCICE N 4 :====

Entrer un entier : 64
64 = 2 puissance 6

====EXERCICE N 4 :====

Entrer un entier : 60
60 n'est pas une puissance de 2.
Entrer un entier : 16
16 = 2 puissance 4

```

/*
Exercice n°5
Ecrire un programme en C qui permet de determiner le nombre de triangles equilateral
inscrits que peut contenir un triangle de cote A>=1.
Le programme s'arrete lorsque le cote du triangle inscrit est < 1.
Exemple : Pour A = 10, le programme affiche :
    Le triangle A=10 peut contenir 1 triangle avec A=10
    Le triangle A=10 peut contenir 4 triangles avec A=5
    Le triangle A=10 peut contenir 16 triangles avec A=2.5
    Le triangle A=10 peut contenir 64 triangles de cote A=1.25
La solution doit utiliser les sous programmes
*/

```

```

#include <stdio.h>
//Saisir le coté d'un triangle avec A>=1=====
float saisie()
{
    float A;
    do{
        printf("Entrez le cote A d'un triangle : ");
        scanf("%f", &A);
    }while(A<=1);
    return A;
}
//Calculer le nombre de triangles pour une décomposition k=i=====
int Nb_triangles(int k) {
    return (k * k);
}
//Afficher le nombre de triangles equilateral inscrits=====
void Affiche(float A)
{
    int i;
    float cote=A;
    i=1;
    while(A>=1) {
        printf(" Le triangle A=%.2f peut contenir Nb=%d triangles avec
A=%.2f \n",cote,Nb_triangles(i),A);
        //i++;
        i=2*i;
        A=cote/i;
    }
}

```

```
//=====
int main() {
    float A;
    printf("\n====EXERCICE N 5 :==== \n\n");
    A=saisie();
    Affiche(A);
    //float cote=A;
    //Determiner et Afficher le nombre de triangles equilateral inscrits
    /* int i=1;
    while(A>=1)
    { printf("Le triangle A=%.2f peut contenir %d triangles avec
A=%.2f\n",cote,Nb_triangles(i),A);
        i++;
        //i=2*i;
        A=cote/i;
    } */
    return 0;
}
```

```
/*
Exercice n°6
Deux nombres a et b sont dits amis, si la somme des diviseurs propres (ses diviseurs, sauf
lui-même) du premier nombre est égale au deuxième nombre, et vice-versa. Exemple : a = 220
et b = 284 sont deux nombres amis
Les diviseurs propres de 220 sont :1, 2, 4, 5, 10, 11, 20, 22, 44, 55 et 110. Leur somme
est 284 Les diviseurs propres de 284 sont 1, 2, 4, 71 et 142. Leur somme est 220
Ecrire en C un programme qui permet d'afficher tous les nombres amis compris entre 1 et
1000.
*/
```

```
#include <stdio.h>
//Determiner la somme de diviseur d'un entier=====
int somme_div(int nb)
{
    int i,s=1;
    for(i=2;i<=nb/2;i++)
        if(nb%i==0)
            s=s+i;
    return s;
}
//Déterminer si deux nombres sont amis ou pas=====
bool ami(int nb1,int nb2)
{
    return(somme_div(nb1)==nb2 && somme_div(nb2)==nb1);
    /*//écriture simplifié de ceci
    if(somme_div(nb1)==nb2 && somme_div(nb2)==nb1)
        return true;
    else return false;*/
}
//=====
int main() {
    int i,j;
    printf("\n====EXERCICE N 6 :==== \n\n");
    printf("Les nombres amis compris entre 1 et 1000:\n");
    for (i = 2; i <= 1000; i++) {
        j = somme_div(i);
        if (i<j && somme_div(j) == i) //i<j pour ne pas avoir les nombres qui se répètent
        {
            printf("%d et %d sont deux nombres amis\n", i, j);
        }
    }
    return 0;
}
```

====EXERCICE N 6 :====

Les nombres amis compris entre 1 et 1000:
220 et 284 sont deux nombres amis

/*

Exercice n°7

Ecrire en C un programme qui permet d'afficher tous les nombres d'Armstrong compris entre 1 et 1000.

On rappelle qu'un nombre d'Armstrong est égale à la somme des cubes de ses chiffres.

Exemple : $153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$ est un nombre de Armstrong

*/

```
#include <stdio.h>
//Determiner le somme de diviseur d'un entier=====
int somme_cube(int nb)
{
    int q,s,r;
    s=0;
    q=nb;
    while(q!=0)
    {
        r=q%10;
        s+=r*r*r;
        q=q/10;
    }
    return s;
}
//determiner si le nombre est Armstrong ou pas=====
bool Armstrong(int nb)
{
    return(nb==somme_cube(nb));
}
//=====
int main() {
    int i;
    printf("\n====EXERCICE N 7 :==== \n\n");
    printf("Liste des nombres Armstrong entre 1 et 1000:\n");
    for (i = 2; i <= 1000; i++) {
        if (Armstrong(i))
        {
            printf("%d \n", i);
        }
    }
    return 0;
}
```

====EXERCICE N 7 :====

Liste des nombres Armstrong entre 1 et 1000:

153

370

371

407